Lecture 1: Introduction

1 Motivating Computational Questions

- Given two integers n, m, can we *compute* their greatest common divisor?

  Yes. The Euclidean algorithm solves this.

- Given a positive integer n, can we *decide* whether or not n is prime?

  Yes. Check all $1 < m < n$ and see whether any m divides n (equivalently, whether $\gcd(n, m) = m$ for some such m). This may take a while.

2 The Big Question

What precisely do we mean when we say we can *compute* a function or *decide* a problem?

2.1 First Attempt at an Answer

We mean there is an *algorithm*, i.e., an *effective procedure*.

But then: how do we define what an effective procedure is?

3 Characteristics of an Algorithm

Some characteristics we look for in an algorithm:

1) A finite list of instructions.
2) The procedure is never vague about what to do next.
3) No intuition is required by the user.
4) The procedure may use scratch work.
5) Whenever the procedure gives an answer, it terminates after a finite number of steps.

In modern language, we often summarize this by saying the procedure can be implemented by a computer.

4 Why a Formal Definition is Needed

Consider the question: "Is f a computable function?"

If the answer is yes, we can show it by describing an algorithm and proving that it works.

If the answer is no, it is less clear how to prove non-computability directly.

Therefore, we need a formal mathematical definition of what it means for f to be computable.

5 An Example of an Undecidable Problem

Hilbert's 10th Problem. Is there an algorithm which, given a Diophantine equation, determines whether or not the equation has an integer solution?

A *Diophantine equation* is a multivariable polynomial equation with integer coefficients; for example,

$$x^2 + y^2 = z^2.$$

This equation does have integer solutions: $x = 3,\ y = 4,\ z = 5$ is one.

Answer to the 10th problem: No. The Diophantine integer-solution problem is *undecidable*.

But to prove this negative answer, mathematicians had to find the correct mathematical definition of *undecidability* and justify (philosophically) that it is the "correct" definition.

6 Course Outline

Two parts of our course:

Part 1:
Provide mathematical definitions of *computable function* and *decidable problem*, and establish their basic properties.

Part 2:
Start exploring the world of the uncomputable and the undecidable.

7 Mathematical Preliminaries

The set of natural numbers is

$$\mathbb{N} := \{0, 1, 2, 3, \dots\}.$$

For a natural number $n > 0$,

$$\mathbb{N}^n := \{(x_0, x_1, \dots, x_{n-1}) : x_0, \dots, x_{n-1} \in \mathbb{N}\},$$

the set of n-tuples of $\mathbb{N}$. We will almost always start our indexing at 0.

7.1 Relations

Let $n > 0$ be an integer. An *n-ary relation* on $\mathbb{N}$ is a subset $R \subseteq \mathbb{N}^n$.

Notation. For $\vec{x} = (x_0, \dots, x_{n-1}) \in \mathbb{N}^n$,

$$\begin{aligned} R(\vec{x}) &\iff \vec{x} \in R && \text{“}R\text{ holds at }\vec{x}\text{”} \\ \neg R(\vec{x}) &\iff \vec{x} \notin R && \text{“}R\text{ does not hold at }\vec{x}\text{”} \end{aligned}$$

Example. Divisibility is a binary relation on $\mathbb{N}$:

$$R(x, y) \iff_{\text{df}} x \text{ divides } y \iff (\exists\, k \in \mathbb{N})\ y = k \cdot x.$$

So $R(2, 4)$ and $R(3, 9)$, but $\neg R(4, 2)$ and $\neg R(2, 7)$.

When $n = 1$, $R \subseteq \mathbb{N}^1$ is called a *unary relation* on $\mathbb{N}$, and is really the same thing as a subset of $\mathbb{N}$.

Example. Define a unary relation P on $\mathbb{N}$ by

$$P(x) \iff_{\text{df}} x \text{ is prime.}$$

As a set, $P = \{x \in \mathbb{N} : x \text{ is prime}\}$.

Lecture 2: Partial Functions

8 Partial Functions

Algorithms can *hang* on inputs, i.e., given an input the algorithm may go on forever and never actually give an output. Because of this fact, in computability theory we deal with *partial functions*.

Definition. Let X, Y be sets. A *partial function* f from X to Y is a function whose domain D is a subset $D \subseteq X$ (possibly $D = X$ or $D = \varnothing$) and whose codomain is Y. We write

$$f\colon X \rightharpoonup Y$$

to mean that f is a partial function from X to Y. (The arrow used is $\rightharpoonup$ instead of $\to$.)

Notation. For $x \in X$,

$$\begin{aligned} f(x)\downarrow &\iff {}_{\mathrm{df}} x \text{ is in the domain of } f \qquad \text{“}f(x) \text{ converges”} \\ f(x)\uparrow &\iff {}_{\mathrm{df}} x \text{ is not in the domain of } f \qquad \text{“}f(x) \text{ diverges”} \end{aligned}$$

Example. $f(x, y) = \dfrac{x}{y}$ defines a partial function from $\mathbb{R}^2$ to $\mathbb{R}$. We have

$$f(x, y)\downarrow \iff y \neq 0.$$

Example. $f(x) = \displaystyle\sum_{n=0}^{\infty} x^n$ defines a partial function from $\mathbb{R}$ to $\mathbb{R}$. We have

$$f(x)\downarrow \iff |x| < 1.$$

We will be concerned almost exclusively about n-ary partial functions on $\mathbb{N}$,

$$f\colon \mathbb{N}^n \rightharpoonup \mathbb{N}.$$

Remark. When $f(\vec{x})\downarrow$ for every $\vec{x} \in \mathbb{N}^n$, we say that f is a *total* function. According to the definition above, total functions are a special type of partial function. So, when we say “let $f\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ be a partial function,” we are *not* ruling out that f is total.

8.1 Equality of Partial Functions

When are two partial functions equal? For $f, g\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$, $f = g$ means that for all $\vec{x} \in \mathbb{N}^n$,

$$f(\vec{x})\downarrow \iff g(\vec{x})\downarrow$$

(i.e., f and g have the same domain), and whenever $\vec{x}$ is in their common domain,

$$f(\vec{x}) = g(\vec{x}).$$

9 Definition Schemes for Partial Functions

Three important ones:

1. Definition by substitution
2. Definition by minimization
3. Definition by primitive recursion

9.1 Definition by Substitution

Roughly speaking, when

$$f(\vec{x}) =_{\text{df}} h\big(g_0(\vec{x}), \ldots, g_{k-1}(\vec{x})\big)$$

we say that f is *defined via substitution* from $h, g_0, \ldots, g_{k-1}$. Here h is k-ary and $f, g_0, \ldots, g_{k-1}$ are n-ary.

Example. Let $h(x_0, x_1, x_2)$ be 3-ary. Define

$$f(x_0, x_1) =_{\text{df}} h(x_1, x_0, x_1).$$

This is a substitution definition using h along with the *projection functions*

$$\pi_0^2(x_0, x_1) =_{\text{df}} x_0, \qquad \pi_1^2(x_0, x_1) =_{\text{df}} x_1.$$

So $f(x_0, x_1) = h\big(\pi_1^2(x_0, x_1),\ \pi_0^2(x_0, x_1),\ \pi_1^2(x_0, x_1)\big)$.
In general, the n-ary projection function onto the ith coordinate ($0 \leq i < n$) is

$$\pi_i^n \colon \mathbb{N}^n \to \mathbb{N}, \qquad \pi_i^n(x_0, \ldots, x_i, \ldots, x_{n-1}) =_{\text{df}} x_i.$$

Definition by substitution works with *partial* functions too. If $h, g_0, \ldots, g_{k-1}$ are partial and

$$f(\vec{x}) = h\big(g_0(\vec{x}), \ldots, g_{k-1}(\vec{x})\big),$$

then

$$f(\vec{x})\downarrow \iff \big(\exists\, y_0, \ldots, y_{k-1}\big)\big[g_0(\vec{x})\downarrow = y_0 \ \wedge\ \cdots\ \wedge\ g_{k-1}(\vec{x})\downarrow = y_{k-1} \ \wedge\ h(y_0, \ldots, y_{k-1})\downarrow\big].$$

9.2 Definition by Minimization

Let $g \colon \mathbb{N}^{n+1} \rightharpoonup \mathbb{N}$ be a partial function. The partial function f *defined via minimization* from g, written

$$f(\vec{x}) = \mu y\big[g(\vec{x}, y) = 0\big],$$

is the n-ary partial function defined by: $f(\vec{x})\downarrow = y$ if and only if

(i) for all $i < y$, $g(\vec{x}, i)\downarrow$ to some nonzero value, and

(ii) $g(\vec{x}, y)\downarrow = 0$.

Example. Let

$$g(x,i) =_{\text{df}} \begin{cases} 0 & \text{if } x \text{ is prime and } i = x^2, \\ 1 & \text{otherwise.} \end{cases}$$

Let $f(x) =_{\text{df}} \mu y\big[g(x,y) = 0\big]$.

What is $f(0)$? Since 0 is not prime, $g(0,i) = 1$ for all i, so $f(0)\uparrow$.

What is $f(3)$? Since 3 is prime,

$$g(3,i) = \begin{cases} 0 & \text{if } i = 3^2 = 9, \\ 1 & \text{otherwise,} \end{cases}$$

so $f(3)\downarrow = 9$. In general,

$$f(x) = \begin{cases} x^2 & \text{if } x \text{ is prime,} \\ \uparrow & \text{otherwise.} \end{cases}$$

Remark. Suppose $f(x) =_{\text{df}} \mu y\big[g(x,y) = 0\big]$. If $g(5,0)\uparrow$ and $g(5,1)\downarrow = 0$, then $f(5)\uparrow$. There is a nice computational intuition for this: to compute $f(5)$ we compute

$$g(5,0),\ g(5,1),\ g(5,2),\ \ldots$$

one by one until we find one that outputs 0. But if $g(5,0)\uparrow$, we get stuck there and never return an output.

Lecture 3: Primitive Recursion

9.3 Definition by Primitive Recursion

Let's start with two basic examples.

(1) The factorial function $y!$ ($y \in \mathbb{N}$) is defined by the recursive equations

$$\begin{cases} 0! = 0, \\ (y+1)! = y! \cdot (y+1). \end{cases}$$

More formally, if $f(y) = y!$ and we define $m_0 := 0$ and $h(t, y) := t \cdot (y+1)$, then

$$\begin{cases} f(0) = m_0, \\ f(y+1) = h\big(f(y),\, y\big). \end{cases}$$

We say "f is defined via primitive recursion (without parameters) from m_0 and h."

(2) $f(x, y) = x + y$ ($x, y \in \mathbb{N}$). Here x will be a parameter of the primitive recursion. f satisfies the recursive equations

$$\begin{cases} f(x, 0) = x, \\ f(x, y+1) = f(x, y) + 1. \end{cases}$$

More formally, if $g(x) := x$ and $h(x, t, y) = t + 1$, then

$$\begin{cases} f(x, 0) = g(x), \\ f(x, y+1) = h\big(x,\, f(x, y),\, y\big). \end{cases}$$

We say "f is defined via primitive recursion from g and h." (Here x could be a tuple $\vec{x}$ of parameters.)

In general, we also allow g, h to be partial functions. Let $n > 0$, $g\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$, $h\colon \mathbb{N}^{n+2} \rightharpoonup \mathbb{N}$ be partial. Then $f\colon \mathbb{N}^{n+1} \rightharpoonup \mathbb{N}$ is *defined by primitive recursion* from g and h if it satisfies

$$\begin{cases} f(\vec{x}, 0) = g(\vec{x}), \\ f(\vec{x}, y+1) = h\big(\vec{x},\, f(\vec{x}, y),\, y\big), \end{cases}$$

where the domain is understood to be given by

$$\begin{aligned} f(\vec{x}, 0)\downarrow &\iff g(\vec{x})\downarrow, \\ f(\vec{x}, y+1)\downarrow &\iff f(\vec{x}, y)\downarrow = t \text{ for some } t, \text{ and } h(\vec{x}, t, y)\downarrow. \end{aligned}$$

10 Recursively Closed Classes

Let $\mathcal{C}$ be a class of partial functions on $\mathbb{N}$ of all sorts of arities.

$\mathcal{C}$ is *recursively closed* if it satisfies all of the following:

1) $\mathcal{C}$ contains the successor function $s\colon \mathbb{N} \to \mathbb{N}$, $s(x) = x + 1$; contains all the projection functions $\pi_i^n(x_0, \ldots, x_i, \ldots, x_{n-1}) := x_i$; and contains all the constant functions of all arities, $c_k^n(x_0, \ldots, x_{n-1}) := k$.

2) $\mathcal{C}$ is closed under definition by substitution; i.e., if $h, g_0, \ldots, g_{k-1} \in \mathcal{C}$, then

$$f(\vec{x}) = h\big(g_0(\vec{x}), \ldots, g_{k-1}(\vec{x})\big) \in \mathcal{C}.$$

3) $\mathcal{C}$ is closed under definition by primitive recursion; i.e., if $g, h \in \mathcal{C}$ and f is defined via primitive recursion from g and h, then $f \in \mathcal{C}$.

4) $\mathcal{C}$ is closed under definition by minimization; i.e., if $g \in \mathcal{C}$ and

$$f(\vec{x}) = \mu y\big[g(\vec{x}, y) = 0\big],$$

then $f \in \mathcal{C}$.

11 Recursive Partial Functions

Definition. A partial function $f\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ is *recursive* if it is a member of every recursively closed class of partial functions.

Put another way, if

$$\mathcal{R} := \bigcap_{\substack{\mathcal{C} \text{ is recursively} \\ \text{closed}}} \mathcal{C},$$

then:

(A) $\mathcal{R}$ is recursively closed, and

(B) $\mathcal{R}$ is contained in every recursively closed class.

Remark. The recursive partial functions are our first "candidates" for the partial functions we should consider to be computable. One can argue that the class of "computable" functions $\mathcal{C}$ is recursively closed, so $\mathcal{R} \subseteq \mathcal{C}$. Whether $\mathcal{C}$ ought to be the smallest recursively closed class is much less clear.

11.1 Examples of Recursive Functions

1. Clearly, the successor s, all projections π_i^n, and all constant functions c_k^n are recursive.

   Note: this includes the identity function $\pi_0^1\colon \mathbb{N} \to \mathbb{N}$, $\pi_0^1(x_0) = x_0$.

2. $f\colon \mathbb{N}^2 \to \mathbb{N}$, $f(x, y) := x + y$ is recursive.

   To see this, first note that

$$\begin{cases} f(x, 0) = \pi_0^1(x), \\ f(x, y + 1) = h\big(x,\ f(x, y),\ y\big), \end{cases}$$

where $h(x,t,y) := s(t)$. Now h is recursive since it is defined via substitution from s and $\pi_1^3(x,t,y) := t$. Thus f is recursive since it is defined via primitive recursion from the recursive functions π_0^1 and h.

3. The predecessor function

$$\mathrm{Pred}(y) = \begin{cases} 0 & \text{if } y = 0, \\ y - 1 & \text{if } y > 0 \end{cases}$$

is recursive, since

$$\begin{cases} \mathrm{Pred}(0) = 0, \\ \mathrm{Pred}(y+1) = \pi_1^2\big(\mathrm{Pred}(y),\, y\big). \end{cases}$$

Lecture 4: More Recursive Partial Functions

12 More Recursive Total Functions

Recall: the recursive partial functions are our first candidates for what the computable partial functions should be.

Theorem. The following are recursive total functions:

1) $\mathrm{Pred}(x) = \begin{cases} x-1 & \text{if } x > 0, \\ 0 & \text{if } x = 0. \end{cases}$

2) Addition $+\colon \mathbb{N}^2 \to \mathbb{N}$.

3) Multiplication $\cdot\colon \mathbb{N}^2 \to \mathbb{N}$.

4) Exponentiation $f(x,y) = x^y$.

All proved by showing they can be defined via primitive recursion from recursive functions. For example,

$$\begin{cases} x^0 = 1, \\ x^{y+1} = x^y \cdot x, \end{cases}$$

assuming we have already shown multiplication is recursive.

12.1 Truncated Subtraction

To avoid negative integers, we define *truncated subtraction* (denoted $\dot{-}$):

$$x \dot{-} y = \begin{cases} x - y & \text{if } x \geq y, \\ 0 & \text{otherwise.} \end{cases}$$

Theorem. Truncated subtraction is recursive.

Proof. Follows from primitive recursion:

$$\begin{cases} x \dot{-} 0 = x, \\ x \dot{-} (y+1) = \mathrm{Pred}(x \dot{-} y). \end{cases} \qquad \square$$

Theorem. There is a recursive total function $f\colon \mathbb{N} \to \mathbb{N}$ such that $f(x) = \frac{x}{2}$ whenever x is even. (We don't care what f does when x is odd.)

Proof. Use minimization. The idea is that $y = \frac{x}{2}$ is equivalent to $2y = x$, i.e., $0 = x \dot{-} 2y$. Define

$$f(x) = \mu y\big[x \dot{-} 2y = 0\big].$$

Here $g(x,y) = x \dot{-} 2y$ is recursive by closure under substitution. It is easy to check that $f(x) = \frac{x}{2}$ when x is even.

Note: What is $f(3)$? We check: $y = 0$: $3 \dot{-} 0 = 3$; $y = 1$: $3 \dot{-} 2 = 1$; $y = 2$: $3 \dot{-} 4 = 0$. So $f(3) = 2$.

Exercise. In general, what is $f(x)$ when x is odd?

Theorem. The following total functions are recursive:

1) $|x - y|$
2) $\max(x, y)$
3) $\min(x, y)$

Proof. Uses clever identities:

$$|x - y| = (x \dot{-} y) + (y \dot{-} x),$$
$$\max(x, y) = \frac{x + y + |x - y|}{2}.$$

Exercise. What about $\min(x, y)$?

12.2 Iterated Addition

Next, we describe how to effectively iterate addition. (Homework: the same can be done for multiplication.)

Theorem. Let $f\colon \mathbb{N}^{n+1} \rightharpoonup \mathbb{N}$ be a recursive partial function. Define $g\colon \mathbb{N}^{n+1} \rightharpoonup \mathbb{N}$ by

$$g(\vec{x}, 0) = 0,$$
$$g(\vec{x}, y + 1) = \sum_{i=0}^{y} f(\vec{x}, i).$$

This converges exactly when $f(\vec{x}, i)\downarrow$ for all $i \leq y$. Then g is recursive.

Proof. By primitive recursion. Note that

$$g(\vec{x}, y + 1) = \sum_{i=0}^{y} f(\vec{x}, i) = \sum_{i=0}^{y-1} f(\vec{x}, i) + f(\vec{x}, y) = g(\vec{x}, y) + f(\vec{x}, y).$$

Formally, let $h(\vec{x}, t, y) := t + f(\vec{x}, y)$, which is recursive. Then

$$\begin{cases} g(\vec{x}, 0) = 0, \\ g(\vec{x}, y + 1) = h\big(\vec{x},\ g(\vec{x}, y),\ y\big). \end{cases} \qquad \square$$

13 Recursive Relations

Definition. An n-ary relation $R \subseteq \mathbb{N}^n$ is *recursive* if its characteristic function

$$\mathrm{Char}_R(\vec{x}) = \begin{cases} 1 & \text{if } R(\vec{x}), \\ 0 & \text{if } \neg R(\vec{x}) \end{cases}$$

is a recursive (total) function.

Idea. If we can compute $\mathrm{Char}_R(\vec{x})$, then we can computably decide whether $R(\vec{x})$ holds: just compute $\mathrm{Char}_R(\vec{x})$; if it equals 1, then "yes"; if it equals 0, then "no."

13.1 Examples of Recursive Relations

(1) $R_{=0}(x) \iff x = 0$ is recursive, since its characteristic function

$$\mathrm{Char}_{R_{=0}}(x) = 1 \dot{-} x$$

is recursive.

(2) $R_<(x, y) \iff x < y$ is recursive, since its characteristic function

$$\mathrm{Char}_{R_<}(x, y) = \min(1,\ y \dot{-} x)$$

is recursive.

Check: $x < y \iff y \dot{-} x \geq 1 \iff \min(1,\ y \dot{-} x) = 1$.

13.2 Closure Under Recursive Total Substitutions

Decidable relations are closed under recursive total substitutions:

Theorem. If $R \subseteq \mathbb{N}^k$ is recursive and $f_0, \ldots, f_{k-1} \colon \mathbb{N}^n \to \mathbb{N}$ are recursive total functions, then

$$Q(\vec{x}) \iff {}_{\mathrm{df}} R\big(f_0(\vec{x}), \ldots, f_{k-1}(\vec{x})\big)$$

is recursive.

Proof. Just check that

$$\mathrm{Char}_Q(\vec{x}) = \mathrm{Char}_R\big(f_0(\vec{x}), \ldots, f_{k-1}(\vec{x})\big)$$

and use that recursive functions are closed under substitution. □

Example. $R(x, y, z) \iff {}_{\mathrm{df}} y < x + z$ is recursive, since

$$R(x, y, z) \iff R_<(y,\ x + z)$$

and $R_<$ is recursive and recursive relations are closed under recursive total substitutions.

Lecture 5: Recursive Relations (cont.)

Last time we defined: an n-ary relation $R \subseteq \mathbb{N}^n$ is recursive if its characteristic function

$$\mathrm{Char}_R(\vec{x}) := \begin{cases} 0 & \text{if } \neg R(\vec{x}), \\ 1 & \text{if } R(\vec{x}) \end{cases}$$

is a recursive (total) function. We saw that $R_{=0}(x) \iff x = 0$ and $R_<(x, y) \iff x < y$ are recursive.

Remark. Recursive relations are *not* closed under recursive *partial* substitutions, but we cannot prove that yet.

Example. The equality relation $R_= = \{(x, y) : x = y\}$ is recursive, since

$$R_=(x, y) \iff |x - y| = 0,$$

"$= 0$" is a recursive relation and $|x - y|$ is a recursive total function.

13.3 Boolean Closure

Theorem. Let $R, Q \subseteq \mathbb{N}^n$ be recursive. Then $\neg R$, $R \wedge Q$, and $R \vee Q$ are all recursive.

Proof. Uses the equalities

$$\begin{aligned} \mathrm{Char}_{\neg R}(\vec{x}) &= 1 \dot{-} \mathrm{Char}_R(\vec{x}), \\ \mathrm{Char}_{R \wedge Q}(\vec{x}) &= \mathrm{Char}_R(\vec{x}) \cdot \mathrm{Char}_Q(\vec{x}), \\ \mathrm{Char}_{R \vee Q}(\vec{x}) &= \min\big(1,\ \mathrm{Char}_R(\vec{x}) + \mathrm{Char}_Q(\vec{x})\big). \end{aligned}$$

(Translate logic to algebra.) □

Example. $R_\leq(x, y) \iff_{\mathrm{df}} x \leq y$ is recursive, since $R_\leq(x, y) \iff x < y \vee x = y$.

13.4 Closure Under Bounded Quantification

Theorem. If $R \subseteq \mathbb{N}^{n+1}$ is recursive, then so are

$$\begin{aligned} P(\vec{x}, y) &\iff_{\mathrm{df}} (\forall\, i \leq y)\ R(\vec{x}, i), \\ Q(\vec{x}, y) &\iff_{\mathrm{df}} (\exists\, i \leq y)\ R(\vec{x}, i). \end{aligned}$$

This is called "closure under bounded quantification."

Proof. We use our iterated addition/multiplication results:

$$\begin{aligned} \mathrm{Char}_P(\vec{x}, y) &= \prod_{i=0}^{y} \mathrm{Char}_R(\vec{x}, i), \\ \mathrm{Char}_Q(\vec{x}, y) &= \min\Big(1,\ \sum_{i=0}^{y} \mathrm{Char}_Q(\vec{x}, i)\Big). \square \end{aligned}$$

Example. The divisibility relation $x \mid y \iff_{\text{df}} x$ divides y is recursive, since

$$x \mid y \iff (\exists\, i \leq y)\ (i \cdot x = y).$$

14 Defining Recursive Partial Functions Using Recursive Relations

Theorem. Let $R(\vec{x}, y)$ be a recursive relation. Define the partial function

$$f(\vec{x}) = \mu y\big[R(\vec{x}, y)\big],$$

i.e., $f(\vec{x}) :=$ the least y such that $R(\vec{x}, y)$ holds ($\uparrow$ if no such y exists). Then f is recursive.

Proof. This is the same as

$$f(\vec{x}) = \mu y\big[1 \dot{-} \operatorname{Char}_R(\vec{x}, y) = 0\big],$$

and $1 \dot{-} \operatorname{Char}_R(\vec{x}, y)$ is a recursive total function. □

Theorem. Let $R(\vec{x})$ be a recursive relation, and let $f, g \colon \mathbb{N}^n \to \mathbb{N}$ be recursive total functions. Then

$$h(\vec{x}) := \begin{cases} f(\vec{x}) & \text{if } R(\vec{x}), \\ g(\vec{x}) & \text{if } \neg R(\vec{x}) \end{cases}$$

is recursive. (We will strengthen this later.)

Proof. Just check that $h(\vec{x}) = f(\vec{x}) \cdot \operatorname{Char}_R(\vec{x}) + g(\vec{x}) \cdot \operatorname{Char}_{\neg R}(\vec{x})$. □

15 Graphs of Partial Functions

Let $f \colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ be an n-ary partial function. The *graph relation* of f is $G_f \subseteq \mathbb{N}^{n+1}$, defined by

$$G_f(\vec{x}, y) \iff f(\vec{x})\downarrow = y.$$

Theorem. If $f \colon \mathbb{N}^n \to \mathbb{N}$ is a *total* function, then f is recursive if and only if G_f is recursive.

Proof. ($\Rightarrow$) Suppose $f(\vec{x})$ is recursive total. Then

$$G_f(\vec{x}, y) \iff f(\vec{x}) = y$$

is recursive, since it is obtained from a recursive total substitution into the recursive relation $=$.

($\Leftarrow$) Suppose G_f is recursive. Then

$$f(\vec{x}) = \mu y\big[G_f(\vec{x}, y)\big],$$

so f is recursive. □

Remarks.

(1) If $f \colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ is partial and G_f is recursive, then f is recursive.

(2) Later we will prove: $f\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ is recursive if and only if G_f is *semirecursive* (to be defined).

Lecture 6: Coding Finite Sequences

16 Coding Finite Sequences

In our set-up, our only "data type" is $\mathbb{N}$ (natural numbers). We also want to be able to talk about finite sequences of numbers $(x_0, x_1, \ldots, x_{n-1})$.

Let $\mathbb{N}^* =$ the set of all finite sequences of natural numbers, including the empty sequence $() = ()$.

Idea: assign each sequence $(x_0, x_1, \ldots, x_{n-1}) \in \mathbb{N}^*$ a numerical code $\langle x_0, \ldots, x_{n-1}\rangle \in \mathbb{N}$. It is important that we do this coding in a way such that from $\langle x_0, \ldots, x_{n-1}\rangle$ we can compute all the basic information about the sequence $(x_0, \ldots, x_{n-1})$ it is coding.

16.1 Primes

Let $p_0, p_1, p_2, \ldots$ list all primes in increasing order ($p_0 = 2$, $p_1 = 3$, $p_2 = 5$, $\ldots$).

Lemma. The function $f(n) = p_n$ is recursive (total).

Proof sketch.

(1) Show $\text{Prime}(x) \iff {}_{\text{df}} x$ is prime is a recursive relation.

(2) Define f via primitive recursion:

$$\begin{cases} f(0) = 2, \\ f(n+1) = \mu y\big[y > f(n) \ \wedge \ \text{Prime}(y)\big]. \end{cases}$$

(Here, $h(t, n) = \mu y\big[y > t \ \wedge \ \text{Prime}(y)\big]$.) □

16.2 The Coding

Definition. If $(x_0, \ldots, x_{n-1})$ is a finite sequence of natural numbers, define

$$\langle x_0, x_1, \ldots, x_{n-1}\rangle = p_0^{x_0+1} \cdot p_1^{x_1+1} \cdots p_{n-1}^{x_{n-1}+1}.$$

This number is the code for $(x_0, \ldots, x_{n-1})$. Also, the code of the empty sequence $()$ is $\langle\rangle = 1$.

Example.

$$\begin{aligned} \langle 2, 1, 0\rangle &= 2^3 \cdot 3^2 \cdot 5^1 = 360, \\ \langle 0\rangle &= 2^1 = 2, \\ \langle 0, 0\rangle &= 2^1 \cdot 3^1 = 6. \end{aligned}$$

Example. Is 50 a code for a sequence? Prime factorization: $50 = 2^1 \cdot 3^0 \cdot 5^2$. The exponent of 3 is 0, which is not of the form $x_i + 1$. So 50 is not a sequence code.

Now, we establish some results which together show that this is a "recursive" coding, in the sense that many natural sequence operations translate to recursive functions on the codes.

16.3 Basic Properties of the Coding

Proposition. Fix any $n \in \mathbb{N}$, $n \geq 1$. Then

$$g\colon \mathbb{N}^n \to \mathbb{N}, \qquad g(x_0, \ldots, x_{n-1}) = \langle x_0, \ldots, x_{n-1} \rangle$$

is a recursive total function.

Proof. Just use that $n \mapsto p_n$ is recursive, exponentiation is recursive, and the definition of g is via substitution. □

Proposition. The unary relation

$$\mathrm{Seq}(u) \iff {}_{\mathrm{df}} u \text{ is a code of a sequence}$$

is recursive.

Proof. $\mathrm{Seq}(u)$ holds exactly when, whenever a prime divides u, all smaller primes also divide u. That is,

$$\mathrm{Seq}(u) \iff (\forall x < u)(\forall y \leq x)\big(\mathrm{Prime}(x) \wedge \mathrm{Prime}(y) \wedge x \mid u \wedge y \leq x \to y \mid u\big).$$

This is a recursive relation $R(x, y, u)$ combined with a recursive relation $Q(x, y, u)$ using the closure properties of recursive relations (bounded quantification, boolean combinations, and divisibility), so Seq is recursive. □

16.4 Recursive Operations on Codes

Theorem. The following are all recursive total functions:

(1) $h(u) := \begin{cases} n & \text{if } u = \langle x_0, \ldots, x_{n-1} \rangle, \\ 0 & \text{otherwise,} \end{cases}$
the length of the sequence coded by u.

(2) $(u)_i := \begin{cases} x_i & \text{if } u = \langle x_0, \ldots, x_i, \ldots, x_{n-1} \rangle, \\ 0 & \text{otherwise,} \end{cases}$
the ith coordinate of the sequence coded by u.

(3) $u \restriction i := \begin{cases} \langle x_0, \ldots, x_{i-1} \rangle & \text{if } u = \langle x_0, \ldots, x_{n-1} \rangle \text{ and } i \leq n, \\ 0 & \text{otherwise,} \end{cases}$
the code of the prefix of length i.

(4) $u * v := \begin{cases} \langle x_0, \ldots, x_{n-1}, y_0, \ldots, y_{m-1} \rangle & \text{if } u = \langle x_0, \ldots, x_{n-1} \rangle \text{ and } v = \langle y_0, \ldots, y_{m-1} \rangle, \\ 0 & \text{otherwise.} \end{cases}$

Proof sketch. For (1) and (2), it is enough to show these functions have recursive graphs:

$$\begin{aligned} \mathrm{lh}(u) = n \iff & \big[n = 0 \wedge (\neg \mathrm{Seq}(u) \vee u = 1)\big] \\ & \vee \big[\mathrm{Seq}(u) \wedge p_{n-1} \mid u \wedge p_n \nmid u\big], \\ (u)_i = y \iff & \big[y = 0 \wedge i \geq \mathrm{lh}(u)\big] \\ & \vee \big[\mathrm{Seq}(u) \wedge p_i^{y+1} \mid u \wedge p_i^{y+2} \nmid u\big]. \end{aligned}$$

Both graphs are recursive using recursiveness of $=$, Seq, 1, and the closure properties of recursive relations.

For (3) and (4), the main idea is to use the auxiliary operation

$$\mathrm{append}(u, k) = \begin{cases} \langle x_0, \ldots, x_{n-1}, k\rangle & \text{if } u = \langle x_0, \ldots, x_{n-1}\rangle, \\ 0 & \text{otherwise.} \end{cases}$$

Then $u * v$ can be seen as repeatedly appending the entries of the sequence coded by v to u, and $u \restriction i$ can be seen as repeatedly appending $(u)_j$ to the empty sequence, for $j < i$. One checks from this description (using previous results) that $u \restriction i$ and $u * v$ are recursive.

17 Applications of Sequence Coding

17.1 Course-of-Values Recursion

Idea: to compute $f(\vec{x}, y+1)$, use all previous values $f(\vec{x}, 0), \ldots, f(\vec{x}, y)$ (think Fibonacci, where $F_{n+2} = F_{n+1} + F_n$).[1]

One way to formalize this is:

$$\begin{aligned} f(\vec{x}, 0) &= g(\vec{x}), \\ f(\vec{x}, y+1) &= h\big(\vec{x}, f(\vec{x}, 0), \ldots, f(\vec{x}, y), y\big). \end{aligned}$$

So $h\colon \mathbb{N}^n \times \mathbb{N}^* \times \mathbb{N} \to \mathbb{N}$.

To fit into our framework, we replace the sequence input $\mathbb{N}^*$ with numerical codes.

Theorem. Let $g\colon \mathbb{N}^n \to \mathbb{N}$, $h\colon \mathbb{N}^n \times \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ be total recursive, and let $f\colon \mathbb{N}^n \times \mathbb{N} \to \mathbb{N}$ be the total function defined by

$$\begin{cases} f(\vec{x}, 0) = g(\vec{x}), \\ f(\vec{x}, y+1) = h\big(\vec{x}, \langle f(\vec{x}, 0), \ldots, f(\vec{x}, y)\rangle, y\big). \end{cases}$$

Then f is recursive.

Proof. Define

$$F(\vec{x}, y) = \langle f(\vec{x}, 0), \ldots, f(\vec{x}, y)\rangle.$$

[1] Sometimes we use the full sequence, sometimes only some of it.

Since $f(\vec{x}, y) = (F(\vec{x}, y))_y$, it is enough to show that F is recursive. Note that

$$\begin{aligned} F(\vec{x}, y+1) &= \langle f(\vec{x}, 0), \ldots, f(\vec{x}, y)\rangle * \langle f(\vec{x}, y+1)\rangle \\ &= F(\vec{x}, y) * \langle h\big(\vec{x}, \langle f(\vec{x}, 0), \ldots, f(\vec{x}, y)\rangle, y\big)\rangle \\ &= F(\vec{x}, y) * \langle h(\vec{x}, F(\vec{x}, y), y)\rangle. \end{aligned}$$

Thus F can be defined via primitive recursion as follows:

$$\begin{cases} F(\vec{x}, 0) = \langle g(\vec{x})\rangle, \\ F(\vec{x}, y+1) = F(\vec{x}, y) * \langle h(\vec{x}, F(\vec{x}, y), y)\rangle, \end{cases}$$

where $G(\vec{x}) = \langle g(\vec{x})\rangle$ and $H(\vec{x}, t, y) = t * \langle h(\vec{x}, t, y)\rangle$ are recursive. So F is recursive, and hence f is recursive. $\square$

Example. The function $f\colon \mathbb{N} \to \mathbb{N}$ defined by

$$\begin{cases} f(0) = 1, \\ f(y+1) = \begin{cases} 1 & \text{if } y = 0, \\ f(y) + f(y-1) & \text{if } y > 0 \end{cases} \end{cases}$$

is recursive by the previous theorem. Here $f(y)$ is the yth Fibonacci number.

Lecture 8: The Unlimited Register Machine (URM)

The URM will give us another way to define the computable partial functions.

18 The Unlimited Register Machine

The URM is a theoretical machine with:

- Infinitely many registers: R_0, R_1, R_2, R_3, ...
- For each register R_i, we have $r_i \in \mathbb{N}$, where r_i is the *content* of register R_i.

19 URM Programs

The URM runs *programs*, which are finite, indexed lists of instructions:

$$I_0,\ I_1,\ \ldots,\ I_{n-1}.$$

20 Types of Instructions

1) **Zero instructions.**
   - Format: $Z(n)$: replace r_n with 0.
2) **Successor instructions.**
   - Format: $S(n)$: replace r_n with $r_n + 1$.
3) **Transfer instructions.**
   - Format: $T(m, n)$: replace r_n with r_m (transfer content of R_m to R_n).
4) **Jump instructions.**
   - Format: $J(m, n, q)$: if $r_m = r_n$, go to instruction I_q; else go to the next instruction on the list.

20.1 Example: Program P

Consider the program P with six instructions:

$$\begin{aligned} &I_0 = J(0,1,5), && I_1 = S(1), && I_2 = S(2), \\ &I_3 = J(0,1,5), && I_4 = J(0,0,1), && I_5 = T(2,0). \end{aligned}$$

Execution trace. Initial configuration: $R_0 = 3$, $R_1 = 2$, $R_2 = 0$, $R_3 = 0$.

R_0	R_1	R_2	R_3	—	Next instruction
3	2	0	0		$I_0 = J(0,1,5)$
3	2	0	0		$I_1 = S(1)$
3	3	0	0		$I_2 = S(2)$
3	3	1	0		$I_3 = J(0,1,5)$
3	3	1	0		$I_5 = T(2,0)$
1	3	1	0		None (halt)

At I_0 and I_3: $r_0 \neq r_1$ at I_0 so we go to I_1; at I_3, $r_0 = r_1 = 3$ so we jump to I_5. Then $T(2,0)$ sets $R_0 := r_2 = 1$, and the program halts.

Remark. We never used $I_4 = J(0,0,1)$ in this run, but when executed, $J(0,0,1)$ always jumps to I_1, since $r_0 = r_0$.

20.2 Program Halting

Example. Consider the program P with two instructions: $I_0 = S(0)$, $I_1 = J(0,0,0)$. This program never *halts* (for any initial state of the registers): it increments R_0 and then jumps back to I_0, looping forever.

In general, we say a program P *halts* when the program instructs the machine to execute an instruction which does not exist (e.g., when the next instruction index is beyond the list $I_0, \ldots, I_{n-1}$).

21 URM-Computable Partial Function

Let P be a program and let $x_0, \ldots, x_{n-1} \in \mathbb{N}$. We write

$$P(x_0, \ldots, x_{n-1})\downarrow \iff_{\text{df}} \text{the computation using } P \text{ and initial state halts,}$$

where the *initial state* is: registers $R_0, R_1, \ldots, R_{n-1}$ contain $x_0, x_1, \ldots, x_{n-1}$ respectively, and $R_n, R_{n+1}, \ldots$ all contain 0. In other words,

	R_0	R_1	R_2	$\cdots$	R_{n-1}	R_n	$R_{n+1} \cdots$
	x_0	x_1	x_2	$\cdots$	x_{n-1}	0	$0 \cdots$

Definition. The n-ary partial function *computed by* P is denoted $f_P^{(n)} \colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ and defined by

$$f_P^{(n)}(\vec{x}) = y \iff_{\text{df}} P(\vec{x})\downarrow \text{ and the content of } R_0 \text{ in the halting state is } y.$$

Definition. A partial function $g \colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ is *URM-computable* if $g = f_P^{(n)}$ for some program P.

21.1 Examples of URM-Computable Functions

Example. The successor function $s\colon \mathbb{N} \to \mathbb{N}$, $s(x) = x + 1$, is URM-computable. Indeed, $s = f_P^{(1)}$ where P is the program with the single instruction $I_0 = S(0)$. With input x in R_0 and all other registers zero, execution does $S(0)$ (so R_0 becomes $x + 1$) and then halts (no instruction I_1). So the content of R_0 in the halting state is $x + 1$, which is the output.

$P(x) \to$	R_0	R_1	R_2	$\cdots$
input	x	0	0	$\cdots$
output	$x+1$	0	0	$\cdots$

Flow: execute $I_0 = S(0)$, then halt.

Remark. An alternate program $\widetilde{P}$ with $I_0 = J(0, 0, 1)$ and $I_1 = S(0)$ also computes s: the jump always goes to I_1, we do $S(0)$, then halt. In general, for a URM-computable partial function, there are infinitely many programs that compute it.

Example. Every constant function $C_k^n\colon \mathbb{N}^n \to \mathbb{N}$ with $C_k^n(\vec{x}) = k$ for all $\vec{x}$ is URM-computable. A program P that computes it is: $I_0 = Z(0)$, $I_1 = S(0)$, $I_2 = S(0)$, $\ldots$, $I_k = S(0)$. So we zero R_0 and then add 1 exactly k times; the content of R_0 at halt is k. This single program P computes the constant k for every arity n:

$$f_P^{(1)} = C_k^1, \qquad f_P^{(2)} = C_k^2, \qquad \ldots$$

(Programs do not come with a specified arity; we interpret the same list of instructions as an n-ary program by taking the first n registers as inputs.)

Example. Every projection function π_i^n $(i < n)$ is URM-computable. It is computed by the program P with the single instruction $I_0 = T(i, 0)$: we copy the content of R_i (the ith input) into R_0, then halt. So the output is the ith coordinate of the input.

22 Equivalence: Recursive = URM-Computable

Let $\mathcal{R}$ = the class of recursive partial functions, and $\mathcal{U}$ = the class of URM-computable partial functions. Our next goal is to sketch the proof of the following theorem.

Theorem. $\mathcal{R} = \mathcal{U}$.

Proof sketch for $\mathcal{R} \subseteq \mathcal{U}$. To show $\mathcal{R} \subseteq \mathcal{U}$, it is enough to show that $\mathcal{U}$ is *recursively closed* (since $\mathcal{R}$ is the smallest recursively closed class). We must check:

- $\mathcal{U}$ contains the successor function, all projection functions, and all constant functions. We have already shown this in the examples above.
- $\mathcal{U}$ is closed under definition by substitution, minimization, and primitive recursion. This requires writing URM programs that implement these definition schemes; the details are tedious URM programming but nothing conceptually interesting.

Lecture 9: $\mathcal{R} = \mathcal{U}$ (Proof Sketch)

Again: $\mathcal{R}$ = class of recursive partial functions, $\mathcal{U}$ = class of URM-computable partial functions.

Theorem. $\mathcal{R} = \mathcal{U}$.

23 Proof Outline

Last time: $\mathcal{R} \subseteq \mathcal{U}$ was shown by proving that $\mathcal{U}$ is recursively closed.

Today: Sketch the proof of $\mathcal{U} \subseteq \mathcal{R}$. To show $\mathcal{U} \subseteq \mathcal{R}$ is conceptually interesting (and harder).

Why it's harder. $\mathcal{U}$ is defined in terms of registers, instructions, and machine steps. $\mathcal{R}$ is defined only in terms of arithmetic (successor, projections, constants, substitution, primitive recursion, minimization). So we have to be able to turn the URM description into statements about natural numbers and arithmetic. This process is often called *arithmetization*.

24 Main Tool: Sequence Coding

The main tool for arithmetizing URM computation is the *sequence coding* we developed earlier (codes $\langle x_0, \ldots, x_{k-1} \rangle$ for finite sequences of natural numbers).

24.1 Coding Instructions

We code URM instructions as numbers (there are many ways to do this). We use # to denote "code of":

- Codes for instruction *types*: $\#Z := 0$, $\#S := 1$, $\#T := 2$, $\#J := 3$.
- Code an instruction by the sequence of its type and arguments:

$$\begin{aligned} \#Z(n) &:= \langle \#Z, n \rangle = \langle 0, n \rangle, \\ \#S(n) &:= \langle \#S, n \rangle = \langle 1, n \rangle, \\ \#T(m,n) &:= \langle \#T, m, n \rangle = \langle 2, m, n \rangle, \\ \#J(m,n,q) &:= \langle \#J, m, n, q \rangle = \langle 3, m, n, q \rangle. \end{aligned}$$

24.2 Codes of Programs

A program P is a sequence of instructions: $P = I_0, I_1, \ldots, I_{n-1}$. The *code of the program* is the code of this sequence of instruction codes:

$$\#P = \langle \#I_0, \#I_1, \ldots, \#I_{n-1} \rangle.$$

24.3 Codes of Machine States

Consider a program $P = I_0, \ldots, I_{n-1}$ in the middle of execution. At any moment we have:

- **Registers:** $R_0, R_1, \ldots, R_m, \ldots$ with current contents $r_0, r_1, \ldots, r_m, \ldots$. (In practice we only care about the *working registers*—those mentioned by P.)
- **Next instruction:** I_t is the instruction to be executed next (t is the program counter). If I_t does not exist (e.g., $t \geq n$), we are in a *halting state*; we may take $\#I_t = 0$ in that case.

The *code for this machine state* is

$$\langle t,\ \#I_t,\ \langle r_0, \ldots, r_m \rangle \rangle,$$

where m is large enough to include all working registers. Given this code, we can *recursively* obtain information about the current state, for example:

- What is currently in R_0?
- What instruction are we about to execute?
- Is this a halting state? (i.e., does I_t exist?)

In other words, these questions correspond to *recursive relations* on natural numbers. So "questions about machines" become "recursive relations about arithmetic."

24.4 The "Next State" Relation

Let $u = \langle t, \#I_t, \langle r_0, \ldots, r_m \rangle \rangle$ and $v = \langle s, \#I_s, \langle \widetilde{r}_0, \ldots, \widetilde{r}_m \rangle \rangle$ be codes of two machine states. We can ask: is v the *next* state after u in the computation with program P? This relation (on codes u, v, and the code of P) is *recursive*. For example, when the current instruction is $Z(1)$ we have conditions such as: $\#I_t = \langle 0, 1 \rangle$, $s = t + 1$, $\widetilde{r}_1 = 0$, and $\widetilde{r}_i = r_i$ for all $i \leq m$ with $i \neq 1$. These are all claims about arithmetic. The cases for the other instruction types (S, T, J) are similar.

25 Kleene T-Predicate

For $n > 0$, define $T_n \subseteq \mathbb{N} \times \mathbb{N}^n \times \mathbb{N}$ by:

$$T_n(e, \vec{x}, y) \iff_{\text{df}}$$

- e is the code of a program P, and
- y codes a *halting computation* of P on input $\vec{x}$, i.e., y codes a sequence of machine states such that:
  (1) The first state $(y)_0$ has registers initialized to $[\vec{x} \mid 0 \mid 0 \cdots]$ (inputs in $R_0, \ldots, R_{n-1}$, rest zero) and next instruction I_0.
  (2) Every subsequent state $(y)_{t+1}$ is obtained from $(y)_t$ by applying the instruction indicated in $(y)_t$.
  (3) The second-to-last state $(y)_{\mathrm{lh}(y)-2}$ is a halting state.
  (4) The content of R_0 in that halting state is $(y)_{\mathrm{lh}(y)-1}$ (the "output" of the computation).

So $T_n(e, \vec{x}, y)$ holds exactly when e codes a program that, on input $\vec{x}$, halts with the computation history coded by y, and the result in R_0 is the last component of y.

Theorem. For any $n > 0$, T_n is recursive.

25.1 Showing $\mathcal{U} \subseteq \mathcal{R}$

We can now show that every URM-computable partial function is recursive. Let $g\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ be URM-computable. Pick a program P that computes it, so $g = f_P^{(n)}$. Let $e = \#P$ (the code of P). Then

$$g(\vec{x}) = (y)_{\mathrm{lh}(y)-1} \quad \text{where } y = \mu y\big[T_n(e, \vec{x}, y)\big].$$

Here $\mu y[T_n(e, \vec{x}, y)]$ is the least y such that $T_n(e, \vec{x}, y)$ holds—i.e., a brute-force search for a code of a halting computation of P on input $\vec{x}$. That y codes a sequence of states whose last component $(y)_{\mathrm{lh}(y)-1}$ is the content of R_0 at halt, i.e., the output $g(\vec{x})$. Extracting $(y)_{\mathrm{lh}(y)-1}$ from y uses the recursive decoding of sequence codes (length and coordinates). Thus g is recursive, since it is defined by minimization of a recursive relation plus finitely many other recursive operations (decoding y and reading off the last coordinate). Hence $\mathcal{U} \subseteq \mathcal{R}$. Equivalently, there is a recursive total function $U\colon \mathbb{N} \to \mathbb{N}$ (extracting the output from a computation code) such that for any computable partial $g\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ there is an $e \in \mathbb{N}$ with $g(\vec{x}) = U\big(\mu y[T_n(e, \vec{x}, y)]\big)$ whenever $g(\vec{x})\downarrow$. In particular, g is recursive.

Lecture 10: Effective Enumerations

26 Recall: T-Predicate and Universal Function

Recall. For each $n > 0$, $T_n \subseteq \mathbb{N} \times \mathbb{N}^n \times \mathbb{N}$ is *recursive*. Roughly: $T_n(e, \vec{x}, y)$ holds iff e codes a program P and y codes a halting computation of P on input $\vec{x}$.

There is also a recursive total function $U\colon \mathbb{N} \to \mathbb{N}$ such that for any computable partial $g\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$, there is an $e \in \mathbb{N}$ with

$$g(\vec{x}) = U\big(\mu y[T_n(e, \vec{x}, y)]\big) \quad (\text{whenever } g(\vec{x})\downarrow).$$

In particular, g is recursive.

27 Terminology: Computable and Decidable

A partial function is *recursive* iff it is URM-computable. Going forward we also refer to such partial functions simply as **computable** (but we will still use "recursive" as well).

Similarly, we say a relation $R \subseteq \mathbb{N}^n$ is **computable** or **decidable** if it is recursive—equivalently, Char_R is URM-computable (i.e., recursive).

28 Uniformly Computable Sequences

Consider an infinite sequence of n-ary computable partial functions: $f_0, f_1, f_2, \ldots$ (so $f_e\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$, $e \in \mathbb{N}$, and each f_e is computable). We say this sequence is **uniformly computable** if the partial function

$$F\colon \mathbb{N} \times \mathbb{N}^n \rightharpoonup \mathbb{N}, \qquad F(e, \vec{x}) := f_e(\vec{x})$$

is also computable.

29 The Universal n-ary Function

For $n > 0$ and $e \in \mathbb{N}$, define

$$\varphi_e^{(n)}(\vec{x}) = U\big(\mu y[T_n(e, \vec{x}, y)]\big).$$

So $\varphi_0^{(n)}, \varphi_1^{(n)}, \ldots$ lists all computable n-ary partial functions (with repeats).

Note. If $f = \varphi_e^{(n)}$, we call e a **code** (or **program**) for f.

This sequence is in fact **uniformly computable**: the partial function

$$\Psi_U^{(n)}(e, \vec{x}) := U\big(\mu y[T_n(e, \vec{x}, y)]\big) = \varphi_e^{(n)}(\vec{x})$$

is computable. So $\Psi_U^{(n)}$ is the *universal n-ary machine* (or universal n-ary function): it takes a code e and an input $\vec{x}$, and outputs $\varphi_e^{(n)}(\vec{x})$ whenever that value is defined.

Example. The function $f\colon \mathbb{N} \to \mathbb{N}$ given by $f(e) = \varphi_e^{(1)}(e)$ is computable. Indeed, $f(e) = \Psi_U^{(1)}(e, e)$, which is obtained from $\Psi_U^{(1)}$ by computable substitution (the projection onto the first coordinate twice).

29.1 Notation

For *unary* partial functions we drop the (1) superscript and write $\varphi_e := \varphi_e^{(1)}$.

30 Conditional Definition (If–Then–Else)

Theorem. If $f, g\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ are computable partial functions and $R \subseteq \mathbb{N}^n$ is computable, then the partial function $h\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ defined by

$$h(\vec{x}) = \begin{cases} f(\vec{x}) & \text{if } R(\vec{x}), \\ g(\vec{x}) & \text{if } \neg R(\vec{x}) \end{cases}$$

is computable.

Proof. Pick codes e_f, e_g such that $f = \varphi_{e_f}^{(n)}$ and $g = \varphi_{e_g}^{(n)}$. Define $v\colon \mathbb{N}^n \to \mathbb{N}$ by

$$v(\vec{x}) = e_f \cdot \mathrm{Char}_R(\vec{x}) + e_g \cdot \mathrm{Char}_{\neg R}(\vec{x}).$$

Then v is total and computable. For each $\vec{x}$, $v(\vec{x})$ equals e_f when $R(\vec{x})$ and e_g when $\neg R(\vec{x})$, so

$$h(\vec{x}) = \varphi_{v(\vec{x})}^{(n)}(\vec{x}) = \Psi_U^{(n)}\big(v(\vec{x}), \vec{x}\big).$$

Hence h is computable, since it is obtained by substitution from the computable total function v, the universal function $\Psi_U^{(n)}$, and the projections. $\square$

Note. In general we do *not* have $h(\vec{x}) = f(\vec{x}) \cdot \mathrm{Char}_R(\vec{x}) + g(\vec{x}) \cdot \mathrm{Char}_{\neg R}(\vec{x})$ when f and g are partial. For example, if $R(\vec{x})$ holds, $f(\vec{x})\downarrow = y$, and $g(\vec{x})\uparrow$, then $h(\vec{x})\downarrow = y$ (we use the $R(\vec{x})$ case), but the right-hand side diverges because $g(\vec{x})$ is undefined. So the algebraic trick that works for total functions does not extend to partial functions; the proof above uses codes and the universal function instead.

31 The S-m-n Theorem

Consider $f\colon \mathbb{N}^m \times \mathbb{N}^n \rightharpoonup \mathbb{N}$, an $(m+n)$-ary computable partial function. For any fixed $\vec{a} \in \mathbb{N}^m$, define the n-ary partial function

$$g_{\vec{a}}(\vec{y}) := f(\vec{a}, \vec{y}).$$

Then $g_{\vec{a}}$ is computable (obtained from f by fixing the first m inputs).

Theorem (S-m-n). There is a computable total function $S\colon \mathbb{N}^m \to \mathbb{N}$ such that for all $\vec{x} \in \mathbb{N}^m$ and $\vec{y} \in \mathbb{N}^n$,

$$\varphi_{S(\vec{x})}^{(n)}(\vec{y}) = f(\vec{x}, \vec{y}).$$

Equivalently, $S(\vec{x})$ is a code for the n-ary partial function $g_{\vec{x}}$.

31.1 Proof sketch (case $m = n = 1$)

Suppose $f(x, y)$ is a computable binary partial function, so $f = \varphi_e^{(2)}$ for some code e. We want a computable total function $S\colon \mathbb{N} \to \mathbb{N}$ such that for every x,

$$\varphi_{S(x)}^{(1)}(y) = f(x, y).$$

Intuitively, $S(x)$ should be a code for a program that, on input y (so initially $R_0 = y$), does:

1) Transfer R_0 to R_1 (so $R_1 := y$).
2) Zero out R_0 and load x into R_0 (so $R_0 := x$).
3) Run a fixed program for f (i.e., the program coded by e) starting from registers $(R_0, R_1) = (x, y)$.

Then the resulting computation agrees with $f(x, y)$.

Technicality. In step (3) we must splice the program for f after the initial "setup" code. If the program for f contains jump instructions, their targets have to be reindexed (shifted) so that they refer to the correct instruction numbers inside the combined program. One shows that this reindexing and concatenation of instruction lists can be done computably on codes, so $x \mapsto S(x)$ is a computable total function.

31.2 Simple application

Claim. There is a computable total function $S'\colon \mathbb{N} \to \mathbb{N}$ such that for every $x \in \mathbb{N}$, $S'(x)$ is a code for the "$+x$" function (the unary function $y \mapsto x + y$).

Proof. Let $f(x, y) = x + y$. Since addition is computable, f is a computable partial function (in fact total). By the S-m-n theorem (with $m = n = 1$), there is a computable total $S'\colon \mathbb{N} \to \mathbb{N}$ such that for all $x, y \in \mathbb{N}$,

$$\varphi_{S'(x)}(y) = f(x, y) = x + y.$$

So $S'(x)$ is a code for the "$+x$" function. $\square$

Lecture 11: More on S-m-n & the Second Recursion Theorem

32 More on the S-m-n Theorem

Theorem (S-m-n). Let $f\colon \mathbb{N}^{m+n} \to \mathbb{N}$ be a computable partial function. Then there is a computable total function $S\colon \mathbb{N}^m \to \mathbb{N}$ such that

$$\varphi^{(n)}_{S(\vec{x})}(\vec{y}) = f(\vec{x}, \vec{y}) \quad \text{for all } \vec{x} \in \mathbb{N}^m,\ \vec{y} \in \mathbb{N}^n.$$

32.1 Proof sketch (case $n = m = 1$, $f(x, y)$)

We want $S(x)$ to be a program for $g(y) := f(x, y)$.

We describe the program $S(x)$ in a way that its instructions can be computed from x:

1) **Transfer R_0 content to R_1.** On input y, initially $R_0 = y$, $R_1 = 0$. After this step: $R_0 = y$, $R_1 = y$.

$$|y|0|0|\cdots \xrightarrow{(1)} |y|y|0|\cdots$$

2) **Load x to R_0.** Zero out R_0, then add 1 repeatedly x times: $Z(0)$, $S(0)$, $S(0)$, $\ldots$ (x many times). After this step: $R_0 = x$, $R_1 = y$.

$$|y|y|0|\cdots \xrightarrow{(2)} |x|y|0|\cdots$$

3) **Run fixed program e_f which computes f.** With $(R_0, R_1) = (x, y)$, run the program for f. Result: $R_0 = f(x, y)$ (other registers arbitrary).

$$|x|y|0|\cdots \xrightarrow{\text{run } e_f} |f(x, y)|*|*|\cdots$$

Easily, there is a computational procedure that, given x, produces these lines of code. A technical detail: one must update instruction-number references in e_f (e.g., jump targets) when splicing it into the combined program—but this reindexing depends computably on x, so $x \mapsto S(x)$ remains a computable total function.

32.2 Concrete example: $x = 3$

For $x = 3$, we want $S(3)$ to be a code for $g(y) = f(3, y)$. The setup instructions are:

$$\begin{array}{lll} I_0 & T(0,1) & \text{(transfer } R_0 \text{ to } R_1) \\ I_1 & Z(0) & \text{(zero } R_0) \\ I_2 & S(0), \quad I_3 = S(0), \quad I_4 = S(0) & \text{(load 3 into } R_0) \end{array}$$

Register evolution: $|y| \to |y|y| \to |3|y|$.

Then $I_5, I_6, \ldots$ are the instructions of e_f (the fixed program for f), but every old instruction-number reference in e_f must have 5 added to it, since those instructions now start at index 5.

So $S(3)$ is the code of the program: $\langle \#T(0,1), \#Z(0), \#S(0), \#S(0), \#S(0)\rangle$ concatenated with the altered e_f.

33 The Second Recursion Theorem

The Second Recursion Theorem allows for the most general type of recursions (more than just primitive recursions).

Theorem (Kleene's Second Recursion Theorem). Let $f(e, \vec{x})$ be an $(n+1)$-ary computable partial function. Then there exists $e^* \in \mathbb{N}$ such that

$$\varphi_{e^*}^{(n)}(\vec{x}) = f(e^*, \vec{x}).$$

The proof is elementary, but a bit mysterious.

Proof. First, define $h\colon \mathbb{N}^{n+2} \rightharpoonup \mathbb{N}$ by

$$h(z, e, \vec{x}) = \varphi_z^{(n+1)}(e, \vec{x}).$$

Then h is computable. By the S-m-n theorem, there is a computable total $S\colon \mathbb{N}^2 \to \mathbb{N}$ such that

$$\varphi_{S(z,e)}^{(n)}(\vec{x}) = h(z, e, \vec{x}) = \varphi_z^{(n+1)}(e, \vec{x}) \quad \text{for all } z, e \in \mathbb{N},\ \vec{x} \in \mathbb{N}^n.$$

Define $g\colon \mathbb{N}^{n+1} \rightharpoonup \mathbb{N}$ by

$$g(e, \vec{x}) = f(S(e,e), \vec{x}).$$

This g is computable, so it has a code e_g (i.e., $\varphi_{e_g}^{(n+1)}(e, \vec{x}) = g(e, \vec{x})$).

Define $e^* = S(e_g, e_g)$. We check that $\varphi_{e^*}^{(n)}(\vec{x}) = f(e^*, \vec{x})$:

$$\begin{aligned}
\varphi_{e^*}^{(n)}(\vec{x}) &= \varphi_{S(e_g,e_g)}^{(n)}(\vec{x}) \\
&= \varphi_{e_g}^{(n+1)}(e_g, \vec{x}) && \text{(by } S\text{-}m\text{-}n\text{, with } z = e_g,\ e = e_g) \\
&= g(e_g, \vec{x}) && (e_g \text{ is a code for } g) \\
&= f(S(e_g, e_g), \vec{x}) && \text{(definition of } g) \\
&= f(e^*, \vec{x}).
\end{aligned}$$

□

33.1 Some applications

1) There is $x^* \in \mathbb{N}$ such that $\varphi_{x^*}(y) = x^* + y$ for all $y \in \mathbb{N}$.

   In other words, x^* is a code for a program that computes the "$+x^*$" function (add x^* to its input).

   Proof. Define $f(x, y) = x + y$, which is computable. By the Second Recursion Theorem, there is $x^* \in \mathbb{N}$ such that $\varphi_{x^*}(y) = f(x^*, y) = x^* + y$ for all $y \in \mathbb{N}$. □

   Remark. It is worth thinking about why the existence of such an x^* is so strange. The

program x^* cannot directly just say "add 1 x^*-many times" or else the program code would be $> x^*$.

2) **Ackermann function.** Define $A(m, n)$ recursively by

$$\begin{aligned} A(0, n) &= n + 1, \\ A(m + 1, 0) &= A(m, 1), \\ A(m + 1, n + 1) &= A(m, A(m + 1, n)). \end{aligned}$$

We know that A is total. In other words: there is exactly one function A that satisfies the above recursive equations. (See Lecture 12 for the proof that A is computable.)

Lecture 12: Second Recursion Theorem & Ackermann

34 Second Recursion Theorem

Theorem (Kleene's Second Recursion Theorem). Let $f\colon \mathbb{N} \times \mathbb{N}^k \rightharpoonup \mathbb{N}$ be a computable partial function. Then there exists $e^* \in \mathbb{N}$ such that

$$\varphi_{e^*}^{(k)}(\vec{x}) = f(e^*, \vec{x}) \quad \text{for all } \vec{x} \in \mathbb{N}^k.$$

35 Application: Ackermann Function is Computable

There is a unique total function $A\colon \mathbb{N}^2 \to \mathbb{N}$ satisfying

$$\begin{aligned} A(0,n) &= n+1, \\ A(m+1,0) &= A(m,1), \\ A(m+1,n+1) &= A(m, A(m+1,n)). \end{aligned}$$

We show A is computable.

Define $F\colon \mathbb{N}^3 \rightharpoonup \mathbb{N}$ by

$$F(e,m,n) = \begin{cases} n+1 & \text{if } m = 0, \\ \varphi_e^{(2)}(\mathrm{Pred}(m), 1) & \text{if } m > 0 \text{ and } n = 0, \\ \varphi_e^{(2)}\big(\mathrm{Pred}(m), \varphi_e^{(2)}(m, \mathrm{Pred}(n))\big) & \text{if } m > 0 \text{ and } n > 0. \end{cases}$$

Then F is computable. By the Second Recursion Theorem, there exists $e^* \in \mathbb{N}$ such that $\varphi_{e^*}^{(2)}(m,n) = F(e^*, m, n)$ for all $m, n \in \mathbb{N}$.

But this means

$$\varphi_{e^*}^{(2)}(m,n) = \begin{cases} n+1 & \text{if } m = 0, \\ \varphi_{e^*}^{(2)}(\mathrm{Pred}(m), 1) & \text{if } m > 0 \text{ and } n = 0, \\ \varphi_{e^*}^{(2)}\big(\mathrm{Pred}(m), \varphi_{e^*}^{(2)}(m, \mathrm{Pred}(n))\big) & \text{if } m > 0 \text{ and } n > 0. \end{cases}$$

$\Rightarrow \varphi_{e^*}^{(2)}$ satisfies the recursive equations of the Ackermann function $\Rightarrow \varphi_{e^*}^{(2)} = A \Rightarrow A$ is computable.

Remark. Much easier than thinking about Ackermann computations directly.

36 Turing Machines

A Turing machine consists of:

- **Infinite tape** (in both directions), divided into squares. Each square can be blank or contain a symbol from an alphabet.
- **Reading head**, which reads one square at a time.
- **Machine state** q, the current state of the machine.

Alphabet: a fixed, finite set of symbols $\{s_0, s_1, \ldots, s_{n-1}\}$. Often the alphabet is just $\{0, 1\}$ with a blank symbol B; this is the binary case.

Finite set of states: $\{q_0, q_1, \ldots, q_{m-1}\}$—the machine has finitely many possible states.

37 Three Types of Instructions

1) **Write and change state:** $q_i\, s_j\, s_k\, q_\ell$

   If in state q_i and reading symbol s_j, then replace s_j with s_k and change to state q_ℓ. (Here s_j, s_k can be B for blank.)

2) **Move right and change state:** $q_i\, s_j\, R\, q_\ell$

   If in state q_i and reading symbol s_j, then move one square to the right and change to state q_ℓ.

3) **Move left and change state:** $q_i\, s_j\, L\, q_\ell$

   Same as (2), but move left instead of right.

Definition. A *Turing machine program* is a finite list of instructions such that for each pair (q_i, s_j), there is at most one instruction that starts with $q_i\, s_j$.

38 Example

Example. Consider Turing machine D with instructions:

- $q_0\, 1\, L\, q_0$: in state q_0 reading 1, move left, stay in q_0
- $q_0\, B\, 1\, q_1$: in state q_0 reading blank B, write 1, change to q_1
- $q_1\, 1\, R\, q_2$: in state q_1 reading 1, move right, change to q_2

Run D on initial configuration: Tape $\ldots B\, 1\, 1\, B\, B\, 1\, B \ldots$, head on the second 1, state q_0.

1) q_0 reads 1 $\to$ apply $q_0\, 1\, L\, q_0$: move left. Head now on first 1, state q_0.
2) q_0 reads 1 $\to$ move left again. Head now on B, state q_0.

3) q_0 reads $B \to$ apply $q_0\, B\, 1\, q_1$: write 1, change to q_1. Tape: $\ldots 1\, 1\, 1\, B\, B\, 1\, B \ldots$, head unchanged.

4) q_1 reads $1 \to$ apply $q_1\, 1\, R\, q_2$: move right, change to q_2. Head on the 1 to the right.

5) q_2 reads $1 \to$ no applicable instruction. **Halts.**

The machine halts because there is no instruction for the pair $(q_2, 1)$.

39 Functions Computed by Turing Machines

Let P be a Turing machine program. The n-ary partial function computed by P is defined as follows.

Given input $\vec{x} = (x_0, \ldots, x_{n-1})$, we start with a tape containing $x_0 + 1$ many 1's, then a blank, then $x_1 + 1$ many 1's, then a blank, and so on, ending with $x_{n-1} + 1$ many 1's (and blanks elsewhere). The head starts on the leftmost 1 in state q_0.

Run the machine according to P. If the machine halts, we define the output to be the number of 1's on the tape in the halting configuration. If the machine does not halt, the function is undefined on $\vec{x}$.

Definition. A partial function $f\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ is *Turing computable* if it is computed by some Turing machine program in this way.

Theorem. A partial function $f\colon \mathbb{N}^n \rightharpoonup \mathbb{N}$ is Turing computable if and only if f is recursive (equivalently, if and only if f is URM-computable).

Lecture 13: Church–Turing Thesis and the Halting Problem

40 Church–Turing Thesis and the Halting Problem

So far, we have established the theory of computable partial functions and decidable relations.

Church–Turing Thesis (informal/philosophical). Our formal mathematical definitions (recursive functions, URM-computable functions, Turing-computable functions, etc.) have precisely captured what we ought to consider to be the *effectively computable* partial functions.

Next we explore the uncomputable. The first (and most important) example of an undecidable relation is the *halting problem* (which actually comes in many different varieties).

Definition (diagonal halting problem). The unary relation $K \subseteq \mathbb{N}$ is defined by

$$K(e) \;\Leftrightarrow_{\text{df}}\; \varphi_e(e)\downarrow.$$

Theorem. K is not decidable. Hence the characteristic function $\text{Char}_K\colon \mathbb{N} \to \mathbb{N}$ is not computable.

Proof (by diagonalization). Suppose toward a contradiction that K is decidable. Define $f\colon \mathbb{N} \to \mathbb{N}$ by

$$f(e) = \begin{cases} \varphi_e(e) + 1 & \text{if } K(e) \text{ holds (i.e., } \varphi_e(e)\downarrow\text{)}, \\ 0 & \text{if } \neg K(e) \text{ holds.} \end{cases}$$

Then f is computable (we can decide $K(e)$ and compute $\varphi_e(e)$ when it converges), so f has a code $e_f \in \mathbb{N}$, i.e., $f = \varphi_{e_f}$. Note that $f = \varphi_{e_f}$ is total.

Now evaluate φ_{e_f} at e_f: we have $\varphi_{e_f}(e_f)\downarrow$ (since φ_{e_f} is total), so $K(e_f)$ holds. By the definition of f,

$$\varphi_{e_f}(e_f) = f(e_f) = \varphi_{e_f}(e_f) + 1 \;\Rightarrow\; 0 = 1.$$

Contradiction. □

40.1 Other variants of the halting problem

Other natural variants of the halting problem are also undecidable:

1) **The full halting problem (for unary partial functions):**

$$H(e, x) \;\Leftrightarrow_{\text{df}}\; \varphi_e(x)\downarrow$$

is undecidable.

Proof. The diagonal halting problem K is a special case of H: $K(e) \Leftrightarrow H(e, e)$ for all e. So if H were decidable, then K would be decidable (decidable relations are closed under total recursive substitutions). Since K is not decidable, H is not decidable.

2) **The full halting problem for n-ary partial functions:**

$$H^{(n)}(e, \vec{x}) \;\Leftrightarrow_{\text{df}}\; \varphi_e^{(n)}(\vec{x})\downarrow$$

is undecidable.

Proof. Recall how URMs work: for any URM program P and $x \in \mathbb{N}$, $f_P^{(1)}(x) = f_P^{(n)}(x, 0, \ldots, 0)$. Hence $\varphi_e(x) = \varphi_e^{(n)}(x, 0, \ldots, 0)$ for all $e, x \in \mathbb{N}$. So

$$K(e) \;\Leftrightarrow\; \varphi_e(e)\downarrow \Leftrightarrow\; \varphi_e^{(n)}(e, 0, \ldots, 0)\downarrow \Leftrightarrow\; H^{(n)}(e, e, 0, \ldots, 0).$$

If $H^{(n)}$ were decidable, then K would be decidable (decidable relations are closed under total recursive substitutions). $\square$

3) **Different effective enumerations** also have undecidable halting problems.

Our effective enumeration $\varphi_0, \varphi_1, \varphi_2, \ldots$ came from URM programs: e codes a URM program. But there are many ways to produce such enumerations (e.g., e could code a Turing machine program; we have $\varphi_0^{(2)}, \varphi_1^{(2)}, \ldots$, $\varphi_0^{(3)}, \varphi_1^{(3)}, \ldots$, etc.).

However we define our effective enumerations, we need them to satisfy:

(1) contain all computable partial functions;

(2) are uniformly computable;

(3) have an S-m-n theorem.

$\Rightarrow$ Then we get the same computability theory. In particular, the halting problem for any such enumeration will be *undecidable*.

Why? The proof that K is undecidable only used: (a) uniform computability of $(\varphi_e)_{e\in\mathbb{N}}$, and (b) that $(\varphi_e)_{e\in\mathbb{N}}$ contains all computable unary functions.

Lecture 14: Index Sets and Rice's Theorem

These concepts will give us many new examples of undecidable relations.

41 Index Sets

Definition. A set $I \subseteq \mathbb{N}$ is an *index set* if for every computable partial function $f\colon \mathbb{N} \rightharpoonup \mathbb{N}$, I either contains *all* codes e for f (where $\varphi_e = f$), or contains *no* codes e for f.

Put another way: I is an index set if

$$(\forall e, z)\big[\varphi_e = \varphi_z \;\Rightarrow\; (e \in I \Leftrightarrow z \in I)\big].$$

Examples. All of the following are index sets:

- $I_0 = \{e : \varphi_e \text{ is total}\}$
- $I_1 = \{e : \mathrm{dom}(\varphi_e) \neq \emptyset\}$
- $I_2 = \emptyset$
- $I_3 = \mathbb{N}$

All index sets can be obtained the following way. Let $\mathcal{C}$ be the set of all unary computable partial functions, and let $A \subseteq \mathcal{C}$. Define

$$I_A := \{e \in \mathbb{N} : \varphi_e \in A\}.$$

Then I_A is an index set.

41.1 Example: K is not an index set

The diagonal halting set $K = \{e : \varphi_e(e) \downarrow\}$ is *not* an index set.

Proof. We use the Second Recursion Theorem. **Claim:** There exists $e^* \in \mathbb{N}$ such that $\mathrm{dom}(\varphi_{e^*}) = \{e^*\}$.

This suffices: then $e^* \in K$ (since $\varphi_{e^*}(e^*) \downarrow$), but for any other z with $\varphi_z = \varphi_{e^*}$, we have $\mathrm{dom}(\varphi_z) = \mathrm{dom}(\varphi_{e^*}) = \{e^*\}$, so $z \neq e^*$ implies $\varphi_z(z) \uparrow$, hence $z \notin K$. Thus K contains e^* but not z even though $\varphi_{e^*} = \varphi_z$, so K is not an index set.

Proof of claim. Define $f\colon \mathbb{N}^2 \rightharpoonup \mathbb{N}$ by

$$f(e, x) = \begin{cases} 0 & \text{if } e = x, \\ \uparrow & \text{otherwise.} \end{cases}$$

Then f is computable (two decidable cases). By the Second Recursion Theorem, there exists $e^* \in \mathbb{N}$ such that $\varphi_{e^*}(x) = f(e^*, x)$ for all x. Thus $\varphi_{e^*}(x) \downarrow = 0$ iff $x = e^*$, so $\mathrm{dom}(\varphi_{e^*}) = \{e^*\}$. $\square$

42 Rice's Theorem

Rice's Theorem. The only decidable index sets are $\emptyset$ and $\mathbb{N}$.

In other words: there are no non-trivial decidable index sets.

Proof. (Uses the Second Recursion Theorem.) Let I be an index set with $I \neq \emptyset$ and $I \neq \mathbb{N}$. Suppose toward a contradiction that I is decidable. Pick $z_0 \in I$ and $z_1 \notin I$.

Define $f\colon \mathbb{N}^2 \rightharpoonup \mathbb{N}$ by

$$f(e,x) = \begin{cases} \varphi_{z_1}(x) & \text{if } e \in I, \\ \varphi_{z_0}(x) & \text{if } e \notin I. \end{cases}$$

Since I is decidable, f is computable. By the Second Recursion Theorem, there exists $e^* \in \mathbb{N}$ such that $\varphi_{e^*}(x) = f(e^*, x)$ for all x, i.e.,

$$\varphi_{e^*}(x) = \begin{cases} \varphi_{z_1}(x) & \text{if } e^* \in I, \\ \varphi_{z_0}(x) & \text{if } e^* \notin I. \end{cases}$$

Case 1: $e^* \in I$. Then $\varphi_{e^*} = \varphi_{z_1}$. Since $z_1 \notin I$ and I is an index set, we must have $e^* \notin I$. Contradiction.

Case 2: $e^* \notin I$. Then $\varphi_{e^*} = \varphi_{z_0}$. Since $z_0 \in I$ and I is an index set, we must have $e^* \in I$. Contradiction.

So I cannot be decidable. □

42.1 Examples

Rice's Theorem gives us many undecidable sets. For example:

1) $\mathrm{Tot} = \{e \in \mathbb{N} : \varphi_e \text{ is total}\}$ is an index set, $\mathrm{Tot} \neq \emptyset$, $\mathrm{Tot} \neq \mathbb{N}$. By Rice's Theorem, Tot is undecidable.

2) By the same reasoning, $\mathrm{Inj} = \{e \in \mathbb{N} : \varphi_e \text{ is injective}\}$ is also undecidable.

Rice's Theorem tells us that there is no *interesting* property P for which we can effectively decide whether the computable partial function computed by a given program e has property P.

43 Index Relations and Rice's Theorem for Relations

We can extend these ideas to relations of arity > 1.

Definition. Let $R \subseteq \mathbb{N}^n$. R is an *index relation* if for all $e_0, \ldots, e_{n-1}, z_0, \ldots, z_{n-1} \in \mathbb{N}$,

$$\big(\varphi_{e_i} = \varphi_{z_i} \text{ for } i = 0, \ldots, n-1\big) \;\Rightarrow\; \big[R(e_0, \ldots, e_{n-1}) \Leftrightarrow R(z_0, \ldots, z_{n-1})\big].$$

Example. $R(e_0, e_1) \Leftrightarrow_{\mathrm{df}} \varphi_{e_0} = \varphi_{e_1}$ is an index relation.

Rice's Theorem for relations. The only decidable n-ary index relations are $\emptyset$ and $\mathbb{N}^n$.

Proof of case $n = 2$. Suppose $R \subseteq \mathbb{N}^2$ is a decidable index relation. For any $e_0 \in \mathbb{N}$, set $I_{e_0} := \{z : R(e_0, z)\}$. Each I_{e_0} is a decidable index set, so by Rice's Theorem, $I_{e_0} = \emptyset$ or $I_{e_0} = \mathbb{N}$. Similarly, for any $z_0 \in \mathbb{N}$, set $J_{z_0} := \{e \in \mathbb{N} : R(e, z_0)\}$. Each J_{z_0} is a decidable index set, so $J_{z_0} = \emptyset$ or $J_{z_0} = \mathbb{N}$.

Suppose $R \neq \emptyset$. We must show $R = \mathbb{N}^2$, i.e., $R(e, z)$ holds for all $e, z \in \mathbb{N}$. Fix $e, z \in \mathbb{N}$. Since $R \neq \emptyset$, there exist e_0, z_0 such that $R(e_0, z_0)$ holds. Then

$$R(e_0, z_0) \;\Rightarrow\; I_{e_0} \neq \emptyset \;\Rightarrow\; I_{e_0} = \mathbb{N} \;\Rightarrow\; z \in I_{e_0} \;\Rightarrow\; R(e_0, z) \text{ holds.}$$

Now

$$R(e_0, z) \;\Rightarrow\; J_z \neq \emptyset \;\Rightarrow\; J_z = \mathbb{N} \;\Rightarrow\; e \in J_z \;\Rightarrow\; R(e, z) \text{ holds, as desired.}$$

$\square$